

TimeTravel Agency — Webapp Interactive

Webapp moderne et immersive pour une **agence de voyage temporel de luxe** (univers fictif), mettant en scène trois destinations à travers le temps et propulsée par l'IA.

Projet final — Supervisé IA — Ynov Campus M1/M2

Membres du groupe

- Ugo Ameslant
- Thomas Barrault
- Torea Tinorua

Liens

- **Webapp déployée** : <https://timetravel-agency-woad.vercel.app>
- **Dépôt GitHub** : <https://github.com/Spyke183/timetravel-agency>

Description

TimeTravel Agency permet à ses clients de **découvrir** trois époques d'exception, d'**interagir** avec un conseiller IA, de **personnaliser** leur voyage via un quiz intelligent et de **réserver** en ligne. L'expérience se veut premium : thème sombre, accents dorés, animations subtiles et navigation fluide « mobile-first ».

Les trois destinations :

| Destination | Époque | Forfait | Durée |

|-----|-----|-----|-----|

| **Paris 1889** | Belle Époque (Tour Eiffel, Exposition Universelle) | 18 900 € | 5 j |

| **Crétacé –65M** | L'ère des dinosaures (safari préhistorique) | 34 500 € | 3 j |

| **Florence 1504** | Renaissance (David de Michel-Ange, Vinci) | 22 400 € | 6 j |

Stack technique

- **Next.js 14** (App Router) + **React 18** + **TypeScript**
- **Tailwind CSS** — design system sur-mesure (thème sombre/doré)
- **Framer Motion** — animations au scroll & micro-interactions
- **lucide-react** — iconographie
- **API Route Next.js** (</api/chat>) — proxy serveur sécurisé pour le LLM (la clé API n'est **jamais** exposée au client)
- **Mistral AI** ([mistral-small-latest](#)) — moteur du chatbot
- **Déploiement** : **Vercel**

Features implémentées

- **Page d'accueil** : hero avec vidéo de fond (optionnelle) + dégradé animé, titre animé mot par mot, double CTA.
- **Présentation de l'agence** : statistiques, arguments clés, storytelling.
- **Galerie des destinations** : 3 cards interactives (hover premium, lazy loading des images) → pages de détail complètes (itinéraire, temps forts, infos pratiques, prix).
- **Agent conversationnel IA (« Chronos »)** : widget flottant bas-droite, thème cohérent, questions suggérées. Répond sur les destinations, prix, conseils et FAQ. **Repli intelligent intégré** : la démo fonctionne même sans clé API.
- **Quiz de personnalisation IA** (automatisation) : 4 questions → algorithme de scoring → **recommandation générée par l'IA**.
- **Formulaire de réservation** : sélection destination + dates + voyageurs, validation côté client, récapitulatif dynamique du prix, écran de confirmation.
- **Animations** : fade-in au scroll, apparition du titre, hover sur les cards, transitions de menu/chat. Respect de `prefers-reduced-motion`.
- **Responsive mobile-first**, SEO (metadata + Open Graph), page 404 personnalisée.

IA utilisées (transparence)

Usage	Outil / Modèle
Génération assistée du code	Cursor + Claude Sonnet (IDE IA recommandé par le cours)
Chatbot conversationnel	Mistral AI — <code>mistral-small-latest</code> (via API, proxy serveur)
Recommandation du quiz	Même endpoint <code>/api/chat</code> (Mistral) avec repli local
Visuels des destinations	(Projet 1) à créditer : ex. Midjourney / DALL·E / Gemini
Vidéos	(Projet 1) à créditer : ex. Runway / Sora / Veo

Le code a été **généré et structuré avec l'assistance d'une IA**, puis relu, testé et ajusté manuellement (build de production validé, scénarios du chatbot testés).

Installation & lancement local

Prérequis : Node.js ≥ 18.

```
# 1. Installer les dépendances
npm install

# 2. (Optionnel) Configurer la clé du chatbot
cp .env.example .env.local
# puis éditez .env.local et collez votre clé : MISTRAL_API_KEY=xxxxxxx
```

```
# Clé gratuite : https://console.mistral.ai -> "API Keys"
```

```
# 3. Lancer en développement
```

```
npm run dev # http://localhost:3000
```

```
# 4. Build de production
```

```
npm run build && npm start
```

Sans clé API, le chatbot bascule automatiquement sur un **moteur de repli**

(connaissances des 3 destinations, prix, FAQ) : la webapp reste pleinement démontrable.

Intégrer vos visuels du Projet 1

Déposez vos fichiers dans `public/destinations/` en respectant ces noms :

```
public/destinations/paris-1889.jpg
public/destinations/cretace.jpg
public/destinations/florence-1504.jpg
public/hero.mp4 (vidéo de fond du hero, optionnelle)
```

Tant qu'un fichier est absent, un **dégradé élégant** s'affiche à la place (jamais d'image cassée).

Déploiement (Vercel)

1. Poussez le projet sur GitHub.
2. Sur vercel.com → **New Project** → importez le dépôt.
3. Dans **Settings** → **Environment Variables**, ajoutez `MISTRAL_API_KEY` (recommandé pour un vrai LLM).
4. **Deploy**. Vercel détecte Next.js automatiquement — aucune configuration requise.

Alternatives : Netlify, Cloudflare Pages, Render (toutes compatibles Next.js).

Structure du projet

```
timetravel-agency/
├── app/
│   ├── layout.tsx # Layout racine (polices, header, footer, chatbot)
│   ├── page.tsx # Page d'accueil (assemble les sections)
│   ├── globals.css # Design system (thème sombre/doré)
│   ├── not-found.tsx # Page 404 personnalisée
│   └── api/chat/route.ts # Endpoint IA sécurisé (Mistral + repli)
├── destinations/[slug]/ # Pages de détail par destination
├── components/ # Header, Hero, Galerie, Quiz, Réservation, Chatbot, ...
├── lib/
│   ├── destinations.ts # Données des 3 destinations (source de vérité unique)
│   └── chatbot.ts # Personnalité IA + base de connaissances + repli
├── public/destinations/ # Vos visuels du Projet 1
└── README.md
```

Cohérence des données : les prix, durées et atouts sont définis **une seule fois** dans `lib/destinations.ts` et réutilisés par l'UI **et** par le prompt du chatbot → les réponses de l'IA sont toujours cohérentes avec le site (critère d'évaluation).

Prompts documentés

Personnalité du chatbot (system prompt — extrait, voir `lib/chatbot.ts`)

Tu es Chronos, l'assistant virtuel de TimeTravel Agency, une agence de voyage temporel de luxe. Ton rôle : conseiller les clients et les aider à choisir la meilleure destination temporelle. Ton ton : professionnel mais chaleureux, passionné d'histoire, enthousiaste sans être familier. Tu connais parfaitement Paris 1889, le Crétacé -65M et Florence 1504 (prix, durées, temps forts). Réponds en français, de façon concise, avec des prix cohérents, sans inventer d'autres destinations.

Prompt de génération initiale (vibe coding)

Crée une webapp Next.js + TypeScript + Tailwind + Framer Motion pour une agence de voyage temporel de luxe. Thème sombre, accents dorés, premium. Sections : Header → Hero (vidéo de fond) → Galerie de 3 destinations (cards) → Présentation agence → Quiz → Réservation → Footer. Widget chatbot flottant en bas à droite. Mobile-first, responsive.

Prompt animations (exercice 2.3)

Ajoute des animations subtiles et élégantes : fade-in progressif des sections au scroll, apparition du titre du hero, hover sur les cards, transitions douces. Durée 0.6-0.8s, easing naturel. Respecte prefers-reduced-motion.

Prompt quiz de personnalisation (exercice 3.2 — Option A)

Crée un quiz interactif de 4 questions pour recommander la destination idéale (type d'expérience, période préférée, ambiance, activité idéale). À la fin, calcule la destination par scoring puis affiche une recommandation personnalisée générée par l'IA, avec une explication.

Prompt intégration du chatbot (exercice 3.1)

Intègre un widget de chatbot en bas à droite : icône flottante, fenêtre au clic, thème sombre/doré, placeholder "Posez-moi vos questions sur les voyages temporels...". Il répond aux questions sur les destinations, les prix, le choix d'une époque et la FAQ. Clé API cachée côté serveur.

Réflexion sur le processus

- **Vibe coding & itération.** Le squelette a été généré en un prompt puis affiné par itérations ciblées (design, animations, cohérence) plutôt que tout régénérer — plus rapide et plus stable.
- **« Context is king ».** Donner au chatbot un rôle clair + une base de connaissances structurée a été décisif pour la **pertinence** et la **cohérence** des réponses.
- **Robustesse pour la démo.** Le repli local du chatbot garantit une webapp fonctionnelle même sans clé API ou en cas de quota dépassé — essentiel pour une présentation/un déploiement public.

- **Une seule source de vérité.** Centraliser les données ([lib/destinations.ts](#)) évite les incohérences entre l'interface et l'IA.
- **Sécurité.** La clé API n'est jamais exposée au navigateur : tous les appels LLM passent par une route serveur Next.js.
- **MVP d'abord.** Priorité aux features qui marchent (chatbot, quiz, réservation) plutôt qu'à la sur-optimisation.

Crédits & licence

- **Framework / libs :** Next.js, React, Tailwind CSS, Framer Motion, lucide-react (licences MIT / open source).
- **IA :** Mistral AI (chatbot), Claude Sonnet via Cursor (assistance au code).
- **Visuels & vidéos :** générés lors du **Projet 1 TimeTravel Agency** — *créditer ici les outils utilisés.*
- **Polices :** Playfair Display & Inter (Google Fonts, SIL Open Font License).

Licence : Projet pédagogique — Ynov Campus M1/M2 Digital & IA. Univers, destinations, personnages et prix **entièrement fictifs**.